
Predicting CVSS and CWE from CVE Descriptions with Fine-Tuned Small Language Models

Tara Dixit

Jessie Dong

Summer Li

Stanford University

{taradixit, jessiedong, summerli}@stanford.edu

Abstract

Security teams prioritize newly disclosed software vulnerabilities by their CVSS vector, severity tier, and weakness class, labels that the U.S. National Vulnerability Database (NVD) supplied for most CVEs until it recently began enriching only high-priority vulnerabilities. As a result, most new vulnerabilities now arrive with a text description and none of the fields on which prioritization depends. To measure how well these fields can be supplied from the description alone, models are trained on CVEs from 2022–2023 in a pinned NVD snapshot and tested on all 36,325 usable CVEs from 2024. The comparison covers TF-IDF classifiers, a 3.8B-parameter language model (Phi-3.5-mini) used zero-shot, and the same model fine-tuned with LoRA, each under two scoring schemes that either predict the CVSS score directly or compute it from the predicted vector. The fine-tuned model is the strongest system on every field, with 69.9% severity accuracy, 70.9% CWE accuracy, a score MAE of 0.85, and an exact CVSS vector on 48.8% of test CVEs, which places it +2.8 to +4.7 points ahead of TF-IDF logistic regression with paired bootstrap intervals that exclude zero. Computing the score from the vector helps logistic regression but trades precision on the CRITICAL tier for recall, the fine-tuned model keeps its own score and vector consistent on 99.8% of CVEs, and the zero-shot model trails both trained systems on every field. Code, configuration files, and per-CVE predictions for every system accompany the paper so that each reported number can be regenerated from the pinned snapshot.

1 Introduction

When software vulnerabilities such as Heartbleed or Log4Shell are disclosed, they receive a CVE identifier and a short text description, but the description alone does not tell a security team how urgently to act. Teams therefore rely on structured *enrichment* consisting of a CVSS v3.1 vector that records how the flaw is exploited and what it affects, the base score (0–10) and severity tier that follow from that vector, and a CWE category for the type of weakness (FIRST, 2019). Scanners, patch managers, and incident responders sort their work queues by these fields, so a CVE that lacks them gives defenders little basis for deciding what to fix first.

For most CVEs, this enrichment was supplied by the National Vulnerability Database (NVD) at NIST, which has not kept pace with the volume of disclosures. A backlog formed

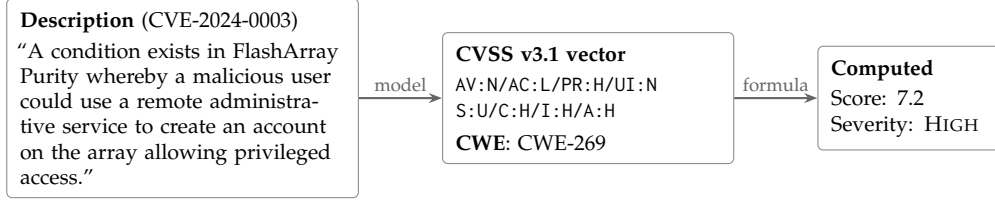


Figure 1: CVE enrichment for CVE-2024-0003. A model reads the description and predicts the eight CVSS components and the CWE, and in the formula variant the score and severity are computed from the predicted vector with the CVSS v3.1 equations so that the three always agree.

in early 2024, and on 15 April 2026 NIST announced that it would enrich only CVEs in CISA’s Known Exploited Vulnerabilities catalog, CVEs in software used by the federal government, and critical software under Executive Order 14028, with every other CVE marked “Lowest Priority – not scheduled for immediate enrichment” (National Institute of Standards and Technology, 2026). For most new CVEs, defenders therefore receive only the description and whatever score the reporting organization chose to supply.

The task studied here, referred to as **CVE enrichment**, is to predict from the description alone the eight CVSS v3.1 vector components, the base score, the severity tier, and the CWE (Figure 1), using only public data and comparing TF-IDF classifiers, a small instruction-tuned language model used zero-shot, and the same model fine-tuned with LoRA.

The data are split by time, with training on CVEs from 2022–2023 and testing on every usable CVE from 2024, so that no model is scored on a period it has already seen. This design avoids the leakage that random splits introduce when near-duplicate descriptions from the same product and month fall on both sides of the split, as they can in several public models. Because the CVSS base score and severity are fixed functions of the eight vector components, the components are predicted and the score is computed with the CVSS formula, following Elbaz et al. (2020) and Shahid and Debar (2021). The comparison then measures both what this computation adds over predicting the score directly and whether a language model’s own score agrees with the vector it writes.

On all 36,325 test CVEs, the fine-tuned model is the strongest system on every field (Figure 2), reaching 69.9% severity accuracy and 70.9% CWE accuracy and predicting the exact CVSS vector for 48.8% of CVEs, +2.8 to +4.7 points ahead of TF-IDF logistic regression. After 2.5 hours of fine-tuning, its scores agree with its own vectors on 99.8% of CVEs, compared with only 3.4% for the same model used zero-shot.

2 Related Work

CVSS vector prediction. Elbaz et al. (2020) predicted each CVSS metric from a bag-of-words description with linear models and derived the score from the predicted vector, with an emphasis on explainable decisions at disclosure time, and CVSS-BERT (Shahid and Debar, 2021) trained one BERT classifier per CVSS v3.1 metric and computed the score with the equations of the specification. Marchiori et al. (2025) compared general-purpose LLMs with embedding-based classifiers for computing CVSS vectors and found the embedding methods stronger on the more subjective impact metrics of confidentiality, integrity, and availability. The present study follows the predict-the-vector design of these works and adds a fine-tuned small language model, a strict temporal split, and a direct comparison with score regression.

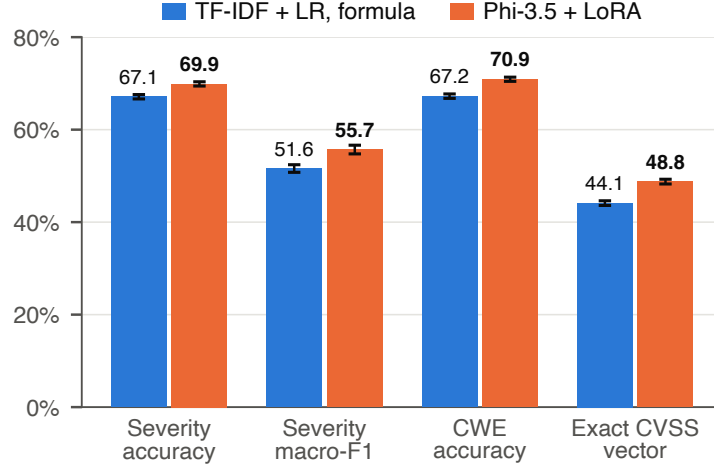


Figure 2: Severity accuracy, severity macro-F1, CWE accuracy, and exact-vector rate for fine-tuned Phi-3.5 and TF-IDF logistic regression with formula scoring on the full 2024 test set (36,325 CVEs), with 95% bootstrap intervals as error bars.

Severity and CWE classification. VLAI (Bonhomme and Dulaunoy, 2025) fine-tunes RoBERTa on more than 600,000 vulnerabilities to predict the severity tier, but its training set spans CVEs from many years including 2024, which leaves it without a fair comparison on the 2024 test set used here (Section 3). For CWE assignment, Aota et al. (2020) used bag-of-words features with random forests, and Mosievskiy (2026) reports that a 125M-parameter RoBERTa is competitive with much larger LLMs.

LLMs for vulnerability triage. Talibzade et al. (2025) found that TF-IDF classifiers outperform general-purpose LLMs with generic prompts on CWE and severity assignment and attribute this to the schematic wording of CVE descriptions, while CVE-LLM (Ghosh et al., 2025) fine-tunes an LLM with a domain ontology to evaluate medical-device vulnerabilities and Torkamani et al. (2025) use LLMs to assign CWEs and severity in a triage tool. For the related task of predicting exploitation, Roytman (2025) reports that frontier LLMs fall well short of the statistical EPSS model and cost far more to run at scale, and industry systems such as VulnWatch (Databricks, 2025) prioritize vulnerabilities with LLMs on hosted infrastructure. The setting studied here differs from these systems in running entirely on local hardware and in evaluating every output on vulnerabilities disclosed after the training data.

3 Task and Data

Task. The input is the English description of a CVE, and the outputs are the eight CVSS v3.1 base metrics (attack vector, attack complexity, privileges required, user interaction, scope, and the confidentiality, integrity, and availability impacts), the base score, the severity tier (CRITICAL, HIGH, MEDIUM, LOW), and the CWE identifier, as in the example of Figure 1.

Source. NVD records are taken from the Fraunhofer FKIE mirror of the NVD JSON feeds (Fraunhofer FKIE, 2026) and pinned to release v2026.10.05-000016, because NVD revises its records over time and a fixed release keeps every reported number reproducible.

Filtering. A CVE is retained if it has an English description of at least 20 characters, a CVSS v3.x base metric with a severity tier, and a specific CWE other than the placeholders

Table 1: Data splits, with severity columns giving the share of CVEs (%) in each tier.

Split	CVEs	Critical	High	Medium	Low	CWEs
Train (2022–23)	40,436	15.4	39.3	43.7	1.7	464
Validation (2022–23)	10,109	15.4	39.3	43.7	1.7	293
Test (2024)	36,325	13.2	36.2	48.6	1.9	512

NVD-CWE-noinfo and NVD-CWE-Other, and when a CVE carries several CVSS entries the primary score assigned by NIST is used. This leaves 86,870 CVEs from 2022–2024, of which 776 use CVSS v3.0 and the remainder v3.1, and the base-score equations of the two versions agree on every one of them.

Temporal split. CVEs with 2022 and 2023 identifiers form a training and validation pool of 50,545 CVEs, which is split 80/20 at random with severity stratification into 40,436 training and 10,109 validation CVEs, while all 36,325 usable CVEs with 2024 identifiers form the test set. Validation data serve only to choose hyperparameters and checkpoints, and Table 1 gives the label distribution of each split.

Label distribution. The LOW tier accounts for under 2% of every split, and CWEs follow a long tail in which the training set contains 464 distinct CWEs and the test set 512, with the 100 most frequent training CWEs covering 91.7% of test CVEs.

CVSS calculator. The CVSS v3.1 base-score equations (FIRST, 2019) are implemented in full, including the floating-point-safe round-up of the specification, and when applied to the true vectors the calculator reproduces the NVD base score and severity for all 86,870 CVEs, so any error in a formula-based score originates in the predicted vector.

Excluded baseline. VLAI (Bonhomme and Dulaunoy, 2025) is a public severity classifier whose training data, the CIRCL/vulnerability-scores dataset, uses a random split over all years, and rows sampled from its training split include CVEs from 2024, 2025, and 2026, so scoring it on the 2024 test set would measure memorization as much as generalization, and it is therefore excluded from the comparison.

4 Methods

4.1 Direct and Formula Scoring

Every system predicts the eight vector components and the CWE, after which the score and severity are produced in one of two ways. In the direct variant, a separate model predicts the score by regression and the severity by classification, or a language model writes both itself, and in the formula variant the CVSS v3.1 equations are applied to the predicted components, so that score, severity, and vector agree by construction. For the TF-IDF systems, the direct score comes from a single ridge regression shared by all three classifiers, so their direct score errors are identical and are reported once.

4.2 TF-IDF Baselines

Each description is represented with TF-IDF over unigrams and bigrams, using sublinear term frequency, a minimum document frequency of 2, and at most 50,000 features, and stop words are kept because words such as “not” and “without” change the meaning of a description. One classifier per output is trained with scikit-learn (Pedregosa et al., 2011), and logistic regression (LR) chooses its regularization strength $C \in \{0.3, 1, 3, 10\}$ by validation

accuracy separately for severity, CWE, and the components, with one C shared by all eight components. Class-balanced LR uses the same setup with class weights inversely proportional to class frequency, and an untuned random forest (RF) uses 300 trees for severity and CWE and 100 trees per component with otherwise default settings. The direct score comes from ridge regression with $\alpha \in \{0.3, 1, 3, 10\}$ chosen by validation MAE. The CWE classifiers learn the 100 most frequent training CWEs plus an OTHER class and are scored against the true CWE, so an OTHER prediction always counts as an error.

4.3 Phi-3.5-mini

The language model is Phi-3.5-mini-instruct, a 3.8B-parameter decoder (Abdin et al., 2024) whose training-data cutoff of October 2023 precedes the test period.

Fine-tuning. The model is fine-tuned with LoRA (Hu et al., 2022) in MLX (Hannun et al., 2023) on a single Apple M5 Pro with 64 GB of unified memory, with adapters of rank 16 (scale 20, dropout 0.05) on the last 16 transformer layers that amount to 12.6M trainable parameters, or 0.329% of the model. Training runs for 5,000 steps at batch size 8, which is one pass over the training set, with a learning rate of 10^{-4} , 100 warm-up steps, and cosine decay to 10^{-5} , and the loss covers only the answer tokens. The 561 training examples (1.4%) longer than 768 tokens are removed to bound memory, while every test CVE is kept regardless of length. The target is a single JSON object that lists the eight components first and then the CWE, the score, and the severity, so the model commits to a vector before it writes a score. Checkpoints at steps 1,000, 2,000, ..., 5,000 are evaluated on the same 1,000 validation CVEs, the checkpoint with the best severity accuracy under formula scoring is kept, and decoding is greedy.

Zero-shot. The same base model without an adapter receives the description and a prompt that lists every field and its allowed values (Appendix A), and because generating verbose zero-shot output for every test CVE is slow, it is evaluated on a random sample of 2,000 test CVEs on which every other system is also rescored.

4.4 Metrics

Severity is evaluated with accuracy and with macro-F1 over the four tiers, which weights LOW as heavily as MEDIUM and so exposes failures on the rare tiers that accuracy hides. CWE prediction is evaluated with top-1 accuracy, the score with mean absolute error (MAE), and the vector with per-component accuracy, the mean fraction of components correct (*partial match*), and the fraction of CVEs with all eight components correct (*exact match*). A language-model output that is not valid JSON counts as wrong on every field and is excluded from the score MAE, with the parse rate reported separately, and 95% confidence intervals come from 1,000 bootstrap resamples of test CVEs.

5 Results

5.1 Main Results

Table 2 reports every system on the 36,325 test CVEs, and the fine-tuned Phi-3.5 model leads on every metric, with 69.9% severity accuracy, 70.9% CWE accuracy, a score MAE of 0.85, and an exact vector on 48.8% of CVEs. Because every system is scored on the same CVEs, the systems can be compared directly with a paired bootstrap against TF-IDF logistic regression with formula scoring. Relative to that baseline, the fine-tuned model changes severity accuracy by +2.8 points (95% CI +2.4 to +3.1), macro-F1 by +4.1 (+3.0 to +5.2), CWE accuracy by +3.7 (+3.4 to +4.0), score MAE by -0.09 (-0.10 to -0.08), and exact-vector rate

Table 2: All systems on the full 2024 test set (36,325 CVEs). Values are percentages except score MAE. $\pm$ gives the half-width of the 95% bootstrap interval. Bold marks the best value in each column. The TF-IDF direct systems share one ridge regression, so their score MAE is identical.

System	Severity		CWE	Score	Vector	
	Acc.	Macro-F1	Acc.	MAE $\downarrow$	Partial	Exact
TF-IDF + LR, direct	67.1 $\pm$ 0.5	47.9 $\pm$ 0.6	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.9 $\pm$ 0.2	44.1 $\pm$ 0.5
TF-IDF + LR, formula	67.1 $\pm$ 0.5	51.6 $\pm$ 0.8	67.2 $\pm$ 0.5	0.94 $\pm$ 0.01	86.9 $\pm$ 0.2	44.1 $\pm$ 0.5
TF-IDF + LR balanced, direct	65.7 $\pm$ 0.5	51.9 $\pm$ 0.8	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.0 $\pm$ 0.2	40.5 $\pm$ 0.5
TF-IDF + LR balanced, formula	67.1 $\pm$ 0.5	53.9 $\pm$ 0.9	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.0 $\pm$ 0.2	40.5 $\pm$ 0.5
TF-IDF + RF, direct	66.9 $\pm$ 0.5	47.1 $\pm$ 0.6	65.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.1 $\pm$ 0.2	41.8 $\pm$ 0.5
TF-IDF + RF, formula	64.0 $\pm$ 0.5	47.8 $\pm$ 0.6	65.2 $\pm$ 0.5	1.03 $\pm$ 0.01	86.1 $\pm$ 0.2	41.8 $\pm$ 0.5
Phi-3.5 + LoRA, direct	69.9 $\pm$ 0.5	55.7 $\pm$ 0.9	70.9 $\pm$ 0.5	0.85 $\pm$ 0.01	87.9 $\pm$ 0.2	48.8 $\pm$ 0.5
Phi-3.5 + LoRA, formula	69.9 $\pm$ 0.5	55.7 $\pm$ 0.9	70.9 $\pm$ 0.5	0.85 $\pm$ 0.01	87.9 $\pm$ 0.2	48.8 $\pm$ 0.5

Table 3: Recall and F1 (%) by true severity tier on the 2024 test set.

System	Critical		High		Medium		Low	
	Rec.	F1	Rec.	F1	Rec.	F1	Rec.	F1
TF-IDF + LR, direct	50.7	51.4	62.2	61.1	77.9	77.1	1.1	2.2
TF-IDF + LR, formula	73.3	56.1	62.6	62.6	71.3	76.8	6.2	10.9
TF-IDF + LR bal., formula	57.1	52.8	61.9	62.2	75.6	77.2	19.2	23.2
Phi-3.5 + LoRA, formula	76.9	59.7	63.6	64.6	75.1	79.0	11.4	19.6

by +4.7 (+4.3 to +5.0). No interval includes zero, so the gains are consistent, although their modest size shows that TF-IDF remains a strong baseline for this task.

5.2 Effect of Formula Scoring

For logistic regression, computing the score from the predicted vector lowers score MAE from 0.98 to 0.94 and raises macro-F1 from 47.9% to 51.6% while accuracy stays at 67.1%, and the gain comes from the two rare tiers (Table 3), with LOW F1 rising from 2.2% to 10.9% and CRITICAL F1 from 51.4% to 56.1% as CRITICAL recall rises from 50.7% to 73.3%.

The improvement carries a cost, because the formula system predicts CRITICAL for 7,755 CVEs where NVD has 4,800, and its precision on that tier falls from 52.1% to 45.4%. It scores 17.7% of test CVEs at exactly 9.8, the score of the most common critical vector (network, low complexity, no privileges, no interaction, high impact), compared with 10.5% in NVD, and its mean signed score error is +0.33, compared with +0.08 for ridge regression. For a triage queue, catching more critical CVEs at the cost of more false alarms may be the preferred trade-off, but it is a choice that each user should make deliberately.

For the random forest, formula scoring lowers accuracy from 66.9% to 64.0% and raises MAE from 0.98 to 1.03, because its component predictions are less accurate and the formula passes those errors through to the score.

The fine-tuned model shows almost no difference between the two modes, since the score it writes matches the CVSS formula for its own vector on 99.8% of test CVEs and the severity it writes matches its own score on 100.0% (Table 5). Trained on targets that list the components first, the model behaves as if it applies the formula to its own vector.

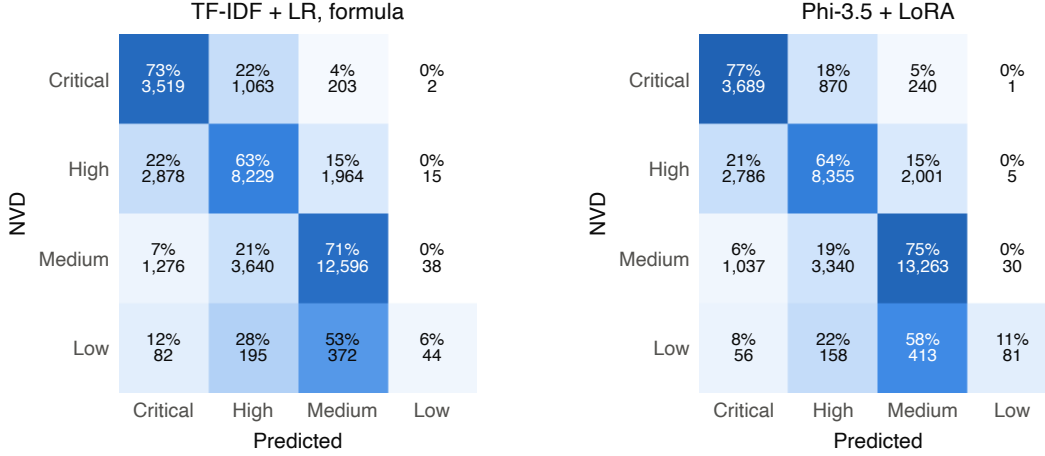


Figure 3: Severity confusion matrices on the 2024 test set, normalized by true tier, for TF-IDF logistic regression with formula scoring (left) and fine-tuned Phi-3.5 (right).

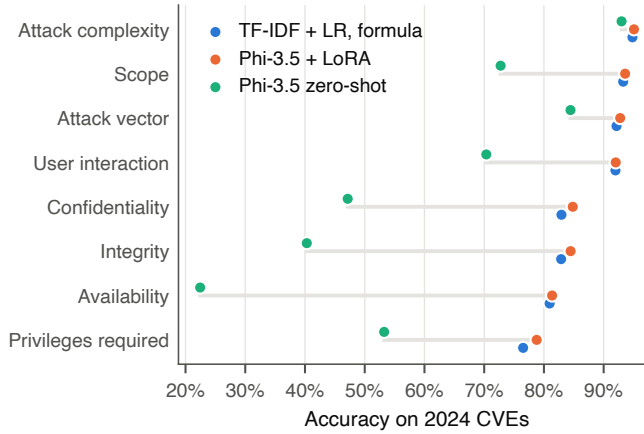


Figure 4: Accuracy per CVSS component on 2024 CVEs, with TF-IDF and the fine-tuned model scored on the full test set and the zero-shot model on its 2,000-CVE sample.

5.3 Error Analysis

Severity tiers. LOW remains the hardest tier for every system (Table 3), since it makes up under 2% of the training data, and both systems most often label a true LOW CVE as MEDIUM. Class-balanced weights raise LOW recall to 19.2% for TF-IDF, the fine-tuned model reaches 11.4% without any rebalancing, and Figure 3 shows that most of the remaining confusion falls between adjacent tiers.

CVSS components. Figure 4 gives accuracy per component, and attack complexity, scope, attack vector, and user interaction exceed 90% for both systems because descriptions usually state them outright. Privileges required is the hardest component (76.5% for TF-IDF and 78.8% for the fine-tuned model), followed by the impact components, and the fine-tuned model gains most on confidentiality (82.9% to 84.8%), integrity (82.9% to 84.5%), and privileges required, the components that depend least on surface keywords. Marchiori et al. (2025) found that general-purpose LLMs fall behind embedding-based classifiers on the impact components, whereas after fine-tuning these components are among those on which the model gains most over TF-IDF.

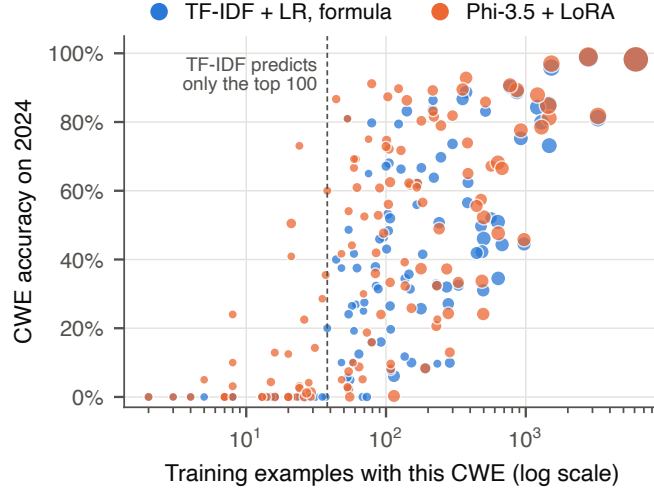


Figure 5: CWE accuracy on the 2024 test set against the number of training examples with each CWE, for every CWE with at least 20 test CVEs and with point size growing with test count.

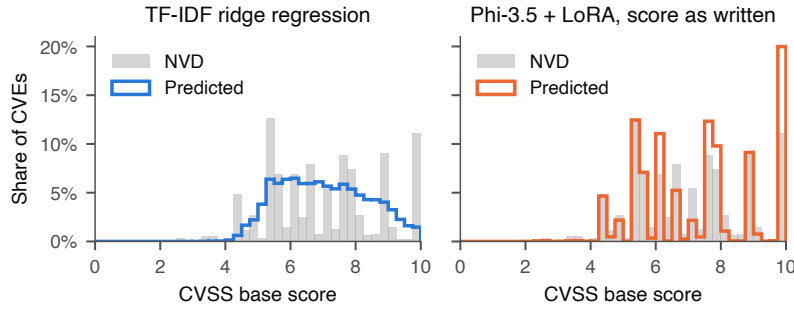


Figure 6: Distribution of predicted and NVD base scores on the 2024 test set in bins of 0.25, for ridge regression (left) and the fine-tuned model with scores as written (right).

Rare CWEs. The TF-IDF classifiers can output only the 100 most frequent training CWEs and therefore score 0% on the 3,023 test CVEs whose CWE is rarer, whereas the fine-tuned model writes the CWE as text and identifies 7.0% of them correctly (Figure 5), and common, explicitly named weaknesses such as cross-site scripting and SQL injection are close to solved by both systems.

Score distribution. Ridge regression produces a smooth spread of scores that NVD never assigns and rarely predicts the extremes (Figure 6), whereas the scores of the fine-tuned model sit on the same discrete values as those of NVD, although it predicts the most common critical score of 9.8 for 20.0% of test CVEs where NVD assigns it to 10.5%. The fine-tuned model predicts CRITICAL for 7,568 test CVEs against 4,800 in NVD, with 48.7% precision, and like formula-scored logistic regression it scores high on average, with a mean signed error of +0.33.

5.4 Zero-Shot Baseline

Without fine-tuning, the same base model is far weaker (Table 4), and although it returned valid JSON for 99.7% of 2,000 random test CVEs, the severity it writes is biased upward. It labels 62.4% of them CRITICAL against 13.3% in NVD and predicts MEDIUM only 9 times al-

Table 4: Comparison on the same random sample of 2,000 test CVEs, on which every system is scored.

System	Severity		CWE	Score	Vector	
	Acc.	Macro-F1	Acc.	MAE ↓	Partial	Exact
TF-IDF + LR, formula	65.3 ±2.1	47.6 ±1.8	66.2 ±2.2	0.99 ±0.06	86.5 ±0.7	42.2 ±2.2
Phi-3.5 zero-shot, direct	20.8 ±1.7	15.4 ±2.4	25.7 ±2.0	1.58 ±0.05	60.5 ±0.7	2.4 ±0.6
Phi-3.5 zero-shot, formula	43.5 ±2.3	26.8 ±1.7	25.7 ±2.0	2.08 ±0.08	60.5 ±0.7	2.4 ±0.6
Phi-3.5 + LoRA, formula	69.6 ±2.0	52.8 ±3.3	68.8 ±2.1	0.87 ±0.05	87.5 ±0.7	48.0 ±2.1

though 47.5% of these CVEs are MEDIUM, so its written severity is correct on only 20.8% of CVEs. Its outputs also contradict themselves, since the score it writes matches the formula for its own vector on only 3.4% of CVEs and the severity it writes matches its own score on 37.0% (Table 5). Computing severity from its vector removes this inconsistency and raises severity accuracy to 43.5%, which remains well below the fine-tuned model (69.6%) and TF-IDF (65.3%) on the same CVEs. Its score MAE, however, worsens from 1.58 to 2.08 because its vectors are often wrong in ways that move the score substantially, so for a zero-shot model formula scoring acts primarily as a consistency fix.

Table 5: Self-consistency of language-model outputs (%), where *Score = formula* is the share of outputs whose written score equals the CVSS v3.1 score of the written vector and *Severity = tier of score* is the share whose written severity matches the tier of the written score.

Model	Valid JSON	Score = formula	Severity = tier of score
Phi-3.5 zero-shot	99.7	3.4	37.0
Phi-3.5 + LoRA	100.0	99.8	100.0

5.5 Training and Inference Cost

Validation accuracy rose at every checkpoint, and the last checkpoint (step 5,000) was the best (Figure 7), which suggests that a further epoch might improve results. Table 6 gives

Table 6: Wall-clock time on a single Apple M5 Pro (64 GB), with inference run in small batches of at most 32 CVEs and without optimization for speed.

Step	Time
All TF-IDF baselines, tuning and test (CPU)	9 min
LoRA fine-tuning, 5,000 steps	2.5 h
LoRA inference, 36,325 test CVEs	4.9 h

wall-clock times, with fine-tuning taking 2.5 hours and scoring the full test year 4.9 hours, compared with 9 minutes for all TF-IDF baselines together.

6 Limitations

Label noise. NVD vectors are treated as ground truth even though analysts disagree, NVD revises its records, and the organizations that report CVEs sometimes score them differently, so some of the reported errors are disagreements that a human reviewer would accept, and the data snapshot is pinned so that the reported numbers stay fixed as the labels change.

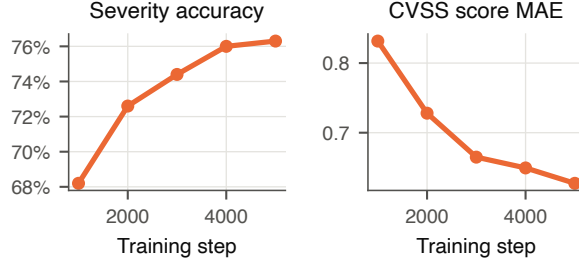


Figure 7: Validation metrics of the fine-tuned model with formula scoring at each saved checkpoint, on 1,000 validation CVEs.

Single split and seed. Evaluation covers a single test year and each model is trained once, so the bootstrap intervals capture the sampling of test CVEs and leave variation from training unmeasured. Only one 3.8B model is fine-tuned, for a single epoch, and its validation accuracy was still rising at the end of training (Figure 7), so larger models or longer training may perform better, and neither large hosted models nor a rebalanced training set for the fine-tuned model were evaluated.

CVSS version. New CVEs increasingly carry CVSS v4.0 vectors, and although the formula approach carries over because v4.0 also computes the score from the vector, it remains untested on v4.0 data.

Zero-shot sample. The zero-shot model is evaluated on 2,000 random test CVEs drawn from the full test set, and the confidence intervals in Table 4 reflect the smaller sample.

Input length. Only English descriptions are used, and descriptions are cut at 2,000 characters, which affects 1.4% of test CVEs.

7 Conclusion

A 3.8B-parameter model fine-tuned for 2.5 hours predicts every NVD enrichment field more accurately than strong TF-IDF baselines on a full year of later CVEs, although the margins are modest and TF-IDF remains a fast and strong baseline that any new method should exceed. Computing the score from the predicted vector keeps outputs consistent and helps a linear model, but it shifts errors toward over-calling CRITICAL, a trade-off that users should choose deliberately, and the rarest severity tier, LOW, remains poorly predicted by every system. Natural next steps are rebalancing the training data, a second training epoch, larger open models, and evaluation on CVSS v4.0.

References

- Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024.
- Masaki Aota, Hideaki Kanehara, Masaki Kubo, Noboru Murata, Bo Sun, and Takeshi Takahashi. Automation of vulnerability classification from its description using machine learning. In *IEEE Symposium on Computers and Communications (ISCC)*, 2020.
- Cédric Bonhomme and Alexandre Dulaunoy. VLAI: A RoBERTa-based model for automated vulnerability severity classification, 2025.
- Databricks. VulnWatch: AI-enhanced prioritization of vulnerabilities. <https://www.databricks.com/labs/vulnwatch>.

-
- databricks.com/blog/vulnwatch-ai-enhanced-prioritization-vulnerabilities, 2025.
- Clément Elbaz, Louis Rilling, and Christine Morin. Fighting n-day vulnerabilities with automated CVSS vector prediction at disclosure. In *Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES)*, 2020. doi: 10.1145/3407023.3407038.
- FIRST. Common vulnerability scoring system v3.1: Specification document. <https://www.first.org/cvss/v3.1/specification-document>, 2019.
- Fraunhofer FKIE. NVD JSON data feeds, release v2026.10.05-000016. <https://github.com/fkie-cad/nvd-json-data-feeds>, 2026.
- Rikhiya Ghosh, Hans-Martin von Stockhausen, Martin Schmitt, George Marica Vasile, Sanjeev Kumar Karn, and Oladimeji Farri. CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025. doi: 10.1609/aaai.v39i28.35139.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Francesco Marchiori, Denis Donadel, and Mauro Conti. Can LLMs classify CVEs? investigating LLMs capabilities in computing CVSS vectors. In *TrustAI/CyberSec Workshop*, 2025.
- Nikita Mosievskiy. Fine-tuning RoBERTa for CVE-to-CWE classification: A 125M parameter model competitive with LLMs, 2026.
- National Institute of Standards and Technology. NIST updates NVD operations to address record CVE growth. <https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth>, April 2026. Accessed 2026-10-05.
- Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Michael Roytman. Benchmarking LLMs on the vulnerability prioritization task. *Empirical Security*. <https://research.empiricalsecurity.com/research/benchmarking-llms>, 2025.
- Mustafizur R. Shahid and Hervé Debar. CVSS-BERT: Explainable natural language processing to determine the severity of a computer security vulnerability from its description. In *20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2021.
- Rustam Talibzade, Idilio Drago, and Francesco Bergadano. On using LLMs for vulnerability classification. In *Proceedings of the 2025 Workshop on Large AI Systems and Models with Privacy and Security Analysis (LAMPS)*, 2025. doi: 10.1145/3733800.3763267.
- Mohammad Jalili Torkamani, Joey NG, Nikita Mehrotra, Mahinthan Chandramohan, Padmanabhan Krishnan, and Rahul Purandare. Streamlining security vulnerability triage with large language models, 2025.

A Prompts

Both prompts are sent as a single user turn through the Phi-3.5 chat template, and descriptions longer than 2,000 characters (1.4% of test CVEs) are cut at that length.

Fine-tuned model (user turn).

You are a cybersecurity expert. Given the CVE description below, predict the NVD enrichment fields.
Return ONLY a JSON object with no extra text.

CVE Description:
<CVE description>

Zero-shot model (user turn).

You are a cybersecurity expert specializing in vulnerability analysis. Given a CVE description, predict the NVD CVSS v3.1 enrichment fields.

CVE Description:
<CVE description>

Respond ONLY with one JSON object with exactly these keys and allowed values:
{
 "attack_vector": "NETWORK"|"ADJACENT_NETWORK"|"LOCAL"|"PHYSICAL",
 "attack_complexity": "LOW"|"HIGH",
 "privileges_required": "NONE"|"LOW"|"HIGH",
 "user_interaction": "NONE"|"REQUIRED",
 "scope": "UNCHANGED"|"CHANGED",
 "confidentiality": "NONE"|"LOW"|"HIGH",
 "integrity": "NONE"|"LOW"|"HIGH",
 "availability": "NONE"|"LOW"|"HIGH",
 "cwe": "CWE-<number>",
 "cvss_score": <number 0.0-10.0>,
 "severity": "CRITICAL"|"HIGH"|"MEDIUM"|"LOW"}
}

B Reproducibility

All code, configuration files, and per-CVE predictions are in the project repository, the NVD snapshot is pinned to FKIE release v2026.10.05-000016, and each step below is a single command that regenerates its part of the paper.

- Data preparation with `download_data.py` and `prepare_splits.py`.
- TF-IDF baselines with `baselines_classical.py`.
- LoRA training with `mlx_lm.lora` and `scripts/lora_phi35.yaml`.
- Checkpoint selection with `select_checkpoint.py`.
- Evaluation with `evaluate_lora.py`.
- Tables, figures, and numbers in the text with `make_tables.py` and `plot_results.py`.